



NIO Selector 技术基础

北京理工大学计算机学院
金旭亮

概述

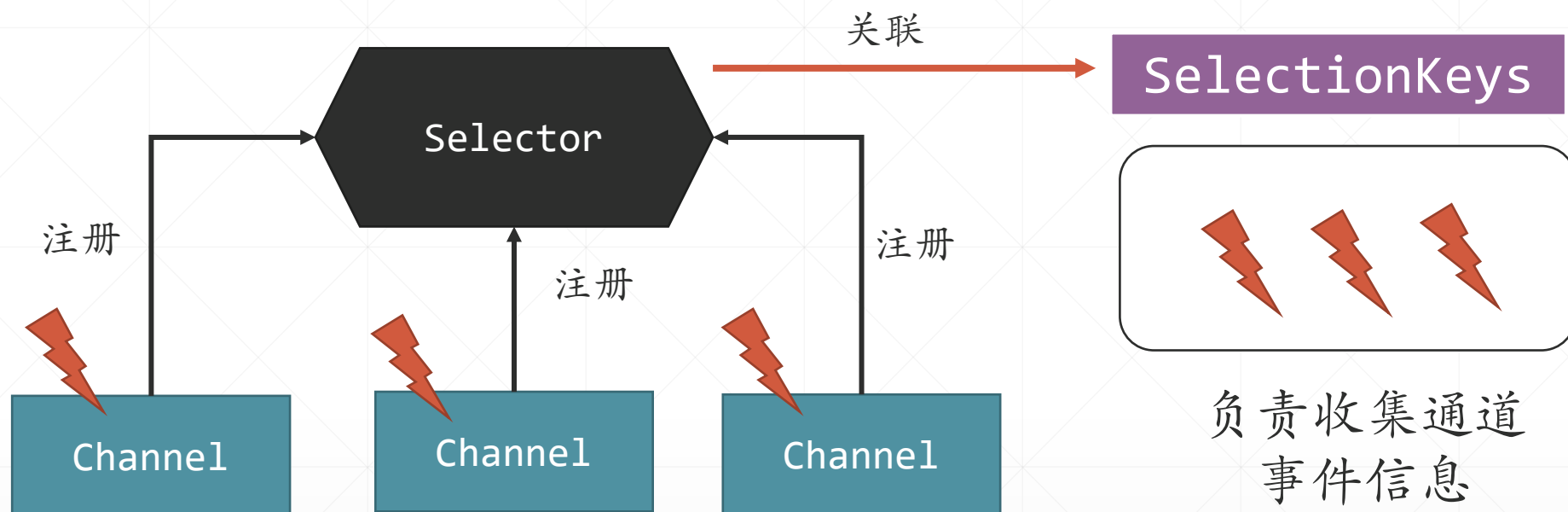


在前面的课程中，介绍过NIO的许多组件，只余下了一个“Selector（选择器）”组件未曾介绍。



Selector组件主要用于网络开发中，用于实现高性能的网络服务应用。

Selector工作原理



一个Selector对应一个线程，可以监控多个通道，每个通道承担特定的任务，比如接收数据或发送数据。这些任务，通过一组OP_XXX常量来表示。

两种类型的通道

ServerSocketChannel

服务端负责监听的通道

SocketChannel

服务端/客户端之间负责数
据传输的通道

一个简单的客户端

```
public class SimpleClient {  
    public static void main(String[] args) {  
        new Thread(() -> {  
            try {  
                Socket socket = new Socket( host: "127.0.0.1", port: 8000);  
                String message = null;  
                while (true) {  
                    message = LocalDateTime.now() + " 你好, Server!";  
                    socket.getOutputStream().write(  
                        message.getBytes(StandardCharsets.UTF_8)  
                    );  
                    System.out.println("发送消息: " + message);  
                    Thread.sleep( millis: 2000);  
                }  
            } catch (IOException e) {  
            }  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            }  
        }).start();  
    }  
}
```

此客户端采用传统编程模型，每隔2秒向Server端发出一条消息。

Server端代码

```
public class NIOServer {  
  
    public static void main(String[] args) throws IOException {  
        //用于接收客户端连接的Selector  
        Selector serverSelector = Selector.open();  
        //用于接收客户端发来的消息的Selector  
        Selector clientSelector = Selector.open();  
        //当有客户端连接时, 接收它, 创建与客户端通讯的通道, 注册到clientSelector上  
        waitConnection(serverSelector, clientSelector);  
        //通过clientSelector监听客户端发来的消息, 并且显示在屏幕上  
        receiveInfoFromClient(clientSelector);  
    }  
}
```

使用open()创建Selector对象

注意我们这里使用了两个Selector。

响应客户端连接请求-1

//等待客户端连接

1 usage

```
private static void waitConnection(Selector serverSelector, Selector clientSelector) {  
    Runnable runnable = () -> {  
        try {  
            //创建用于监听的服务端通道  
            ServerSocketChannel listenerChannel = ServerSocketChannel.open();  
            //设置为非阻塞模式  
            listenerChannel.configureBlocking(false);  
            //绑定8000端口, 进行监听  
            listenerChannel.socket().bind(new InetSocketAddress(port: 8000));  
            //监听通道向Selector注册, 表明对“客户端连接 (OP_ACCEPT)”事件感兴趣  
            listenerChannel.register(serverSelector, SelectionKey.OP_ACCEPT);  
            System.out.println("服务端已启动, 在8000端口监听");  
            while (true) {  
                //处理客户端连接  
                handleConnectRequest(serverSelector, clientSelector);  
            }  
        } catch (Exception e) {  
        }  
    };  
    new Thread(runnable).start();  
}
```

使用open()创建服务端通道

见下页

响应客户端连接请求-2

```
private static void handleConnectRequest(Selector serverSelector, Selector clientSelector) throws IOException {  
    //最长等待10秒, 如果10秒内没有连接, 继续下轮循环  
    if (serverSelector.select( timeout: 10000) > 0) {  
        //获取当前事件列表  
        Set<SelectionKey> set = serverSelector.selectedKeys();  
        //遍历事件列表  
        Iterator<SelectionKey> keyIterable = set.iterator();  
        while (keyIterable.hasNext()) {  
            SelectionKey key = keyIterable.next();  
            //如果是连接事件  
            if (key.isAcceptable()) {  
                try {  
                    //创建与此客户端通讯的通道  
                    SocketChannel clientChannel = ((ServerSocketChannel) key.channel()).accept();  
                    //设置为非阻塞模式  
                    clientChannel.configureBlocking(false);  
                    //通道向Selector注册, 表明对“数据读取 (OP_READ)”事件感兴趣  
                    clientChannel.register(clientSelector, SelectionKey.OP_READ);  
                } finally {  
                    //此事件已经处理完毕, 可以从事件集合中移除  
                    keyIterable.remove();  
                }  
            }  
        }  
    }  
}
```


SelectionKey

SelectionKey是一个关键的对象，它的以下成员非常重要：



`channel()`：用于获取对应的`ServerSocketChannel`或`SocketChannel`



`isXXX()`：用于判断是哪种事件，比如，`isReadable()`，用于表明通道中有数据到达。

接收客户端数据-1

接收客户端数据代码与接收客户端连接类似，都是等待10秒钟，如果此期间“有事发生”，并且此事是“有数据到达”，则利用通道接收数据。

```
private static void receiveInfoFromClient(Selector clientSelector) {
    Runnable runnable = () -> {
        try {
            while (true) {
                if (clientSelector.select( timeout: 10000) > 0) {
                    Set<SelectionKey> set = clientSelector.selectedKeys();
                    var keyIterator : Iterator<SelectionKey> = set.iterator();
                    while (keyIterator.hasNext()) {
                        var key : SelectionKey = keyIterator.next();
                        //只对“数据接收”事件感兴趣
                        if (key.isReadable()) {
                            try {
                                //接收数据并输出
                                receiveMessageAndOutput(key);
                            } finally {
                                keyIterator.remove();
                            }
                        }
                    }
                }
            }
        } catch (IOException e) {
        }
    };
    new Thread(runnable).start();
}
```

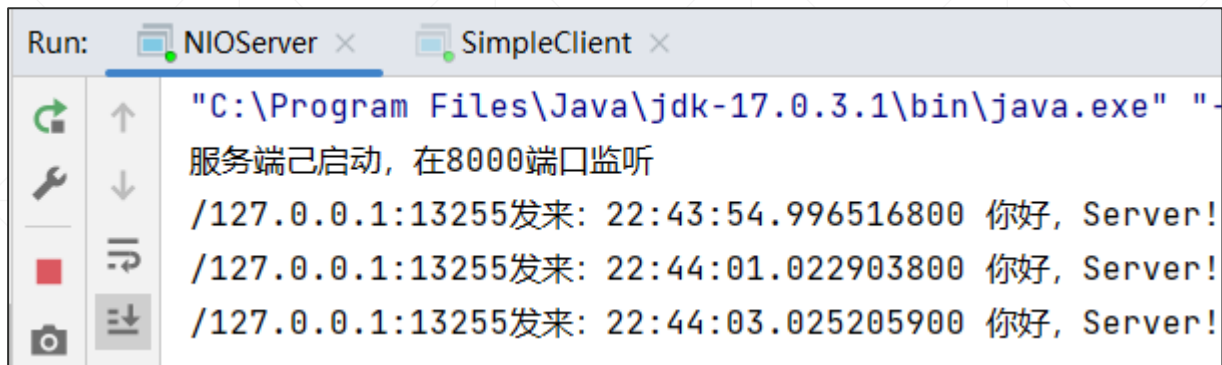
接收客户端数据-2

```
private static void receiveMessageAndOutput(SelectionKey key) throws IOException {  
    SocketChannel clientChannel = (SocketChannel) key.channel();  
    //分配数据缓冲区  
    var byteBuffer : ByteBuffer = ByteBuffer.allocate( capacity: 1024);  
    clientChannel.read(byteBuffer);  
    byteBuffer.flip();  
  
    String clientAddress = String.valueOf(clientChannel.getRemoteAddress());  
    //解码传入的消息并输出  
    String message = StandardCharsets.UTF_8.newDecoder()  
        .decode(byteBuffer).toString();  
    System.out.println(clientAddress + "发来: " + message);  
}
```

标准的NIO通道用法

运行截图

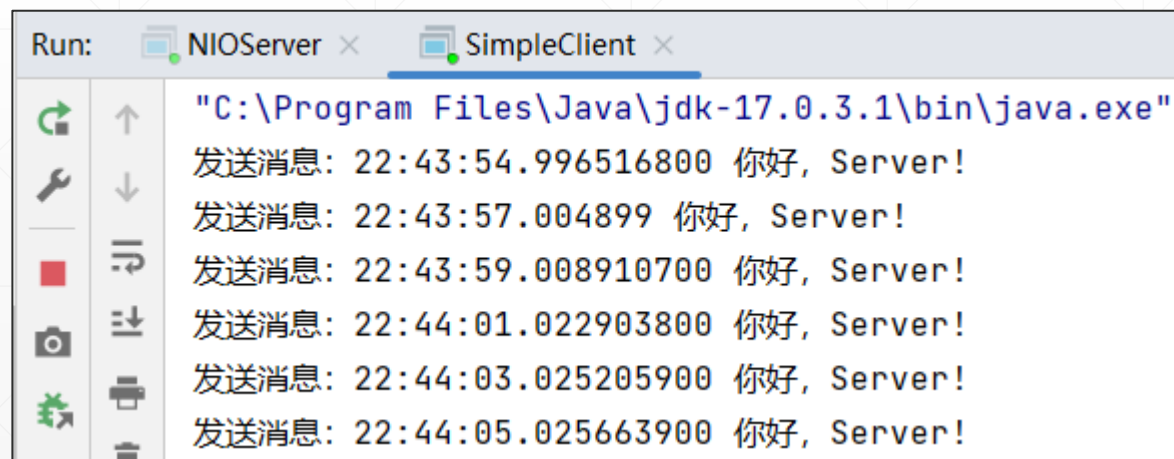
Server端输出



The screenshot shows the Run console for the NIOServer application. The console output is as follows:

```
Run: NIOServer x SimpleClient x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" "-
服务端已启动, 在8000端口监听
/127.0.0.1:13255发来: 22:43:54.996516800 你好, Server!
/127.0.0.1:13255发来: 22:44:01.022903800 你好, Server!
/127.0.0.1:13255发来: 22:44:03.025205900 你好, Server!
```

Client端输出



The screenshot shows the Run console for the SimpleClient application. The console output is as follows:

```
Run: NIOServer x SimpleClient x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
发送消息: 22:43:54.996516800 你好, Server!
发送消息: 22:43:57.004899 你好, Server!
发送消息: 22:43:59.008910700 你好, Server!
发送消息: 22:44:01.022903800 你好, Server!
发送消息: 22:44:03.025205900 你好, Server!
发送消息: 22:44:05.025663900 你好, Server!
```

要点小结



Selector可以监听多个Channel，它是通过SelectionKey来管理这些通道的，它把这些SelectionKey放到了一个集合中。



每个通道都要调用register()方法向Selector注册，声明要它关注哪些事件，这些事件，是通过一组OP_XXX常量进行标识的。



Selector的select()函数是阻塞调用，它负责检查当前注册的通道有无“事件发生”，有则立即返回，无则继续阻塞等待预先指定的时间。



Selector的select()函数返回后，它返回“当前有事”的SelectionKey，通过它就能完成后继的处理工作（比如，通过通道读或写数据）。